

AI/ML PROJECT SUBMISSION



PROJECT OVERVIEW	STUDENT DETAILS
• Project Number: 5 • Task Name: Inventory Management System. • Difficulty: Moderate to advanced.	• Name: Pranav Gajanan Rane • College: Government Polytechnic Gondia • Branch: Computer Engineering • Domain: AI/ML

Project 5: Inventory Management System

The Inventory Management System is a comprehensive stock-control application built in Python. It tracks products by ID, name, category, price, and quantity. The system allows stock in (restocking), stock-out (sales), generates low-stock alerts, and produces inventory reports with total stock value. It uses all core Python data structures and integrates basic file handling for persistent storage.

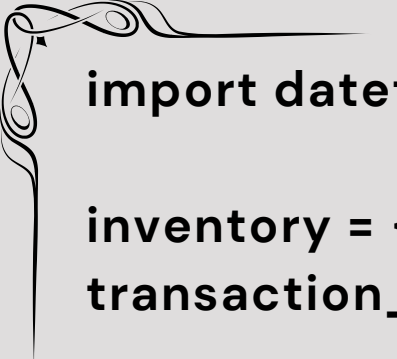
CODE-

```
#-----  
-----  
'''
```

```
Project 5 - Inventory Management System  
Made by Pranav Rane  
'''
```

```
#-----  
-----
```

```
'''Functions Definitions followed by actual system.'''
```



```
import datetime
```

```
inventory = {}
```

```
transaction_log = []
```

```
def load_inventory(): # Fetches the data from  
inventory.txt
```

```
    global inventory
```

```
    try:
```

```
        f = open("inventory.txt", "r")
```

```
        lines = f.readlines()
```

```
        f.close()
```

```
        for line in lines:
```

```
            line = line.strip()
```

```
            if line == "":
```

```
                continue
```

```
            parts = line.split("|")
```

```
            pid = parts[0]
```

```
            inventory[pid] = {
```

```
                "name": parts[1],
```

```
                "category": parts[2],
```

```
                "price": float(parts[3]),
```

```
                "quantity": int(parts[4]),
```

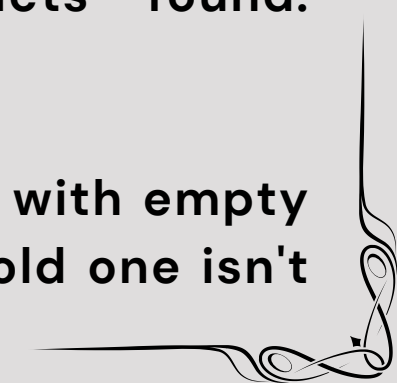
```
                "reorder": int(parts[5])
```

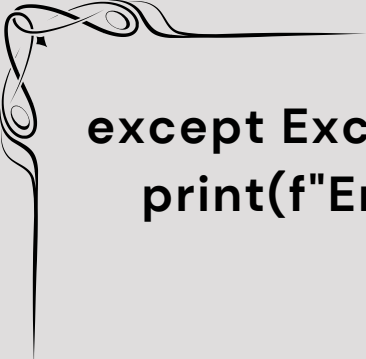
```
            }
```

```
            print(f"Inventory loaded. Products found:  
{len(inventory)}")
```

```
    except FileNotFoundError:
```

```
        print("Inventory file not found. Starting with empty  
inventory.") #Makes new inventory file if old one isn't  
found
```





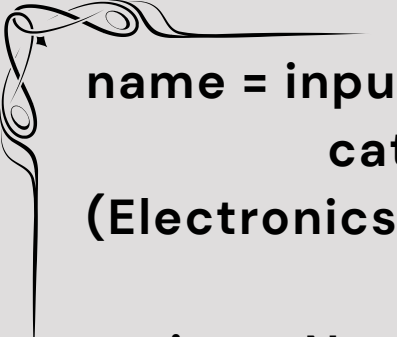
```
except Exception as e:  
    print(f"Error loading file: {e}")
```

```
def save_inventory(): #Loads the data in inventory.txt  
    try:  
        f = open("inventory.txt", "w")  
        for pid, p in inventory.items():  
            line = f"{pid}|{p['name']}|{p['category']}|{p['price']}|  
{p['quantity']}|{p['reorder']}\n"  
            f.write(line)  
        f.close()  
        print("Product added & saved to inventory.txt")  
    except Exception as e:  
        print(f"Error saving file: {e}")
```

```
def log_transaction(pid, ttype, qty): # Keeps logs of  
every transaction being made  
    now = datetime.datetime.now().strftime("%Y-%m-%d  
%H:%M:%S")  
    transaction_log.append((pid, ttype, qty, now))
```

```
def add_product(): # To add product in the stock  
    pid = input("Enter Product ID (e.g. P001):  
").strip().upper()  
    if pid in inventory:  
        print("Product ID already exists. Use a different ID.")  
    return
```



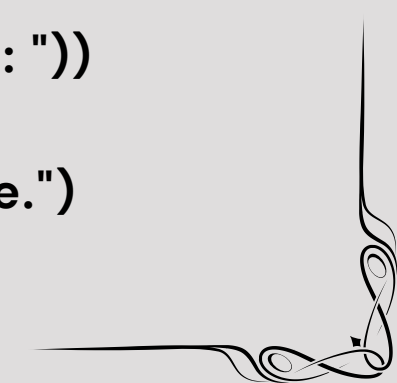


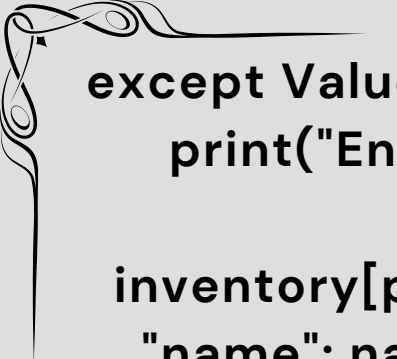
```
name = input("Enter Product Name: ")
category = input("Enter Category
(Electronics/FMCG/Apparel/Stationery): ")
```

```
price = None
while price is None:
    try:
        price = float(input("Enter Unit Price: "))
        if price < 0:
            print("Price cannot be negative!!!")
            price = None
    except ValueError:
        print("Enter a valid number.")
```

```
qty = None
while qty is None:
    try:
        qty = int(input("Enter Initial Quantity: "))
        if qty < 0:
            print("How can quantity be negative SON?")
            qty = None
    except ValueError:
        print("Enter a valid number.")
```

```
reorder = None
while reorder is None:
    try:
        reorder = int(input("Enter Reorder Level: "))
        if reorder < 0:
            print("Reorder level cannot be negative.")
            reorder = None
```





```
except ValueError:
```

```
    print("Enter a valid number.")
```

```
inventory[pid] = {
```

```
    "name": name,
```

```
    "category": category,
```

```
    "price": price,
```

```
    "quantity": qty,
```

```
    "reorder": reorder
```

```
}
```

```
print(f"Product '{name}' added successfully.")
```

```
def stock_in(): #Refreshes with the new stocks.
```

```
    pid = input("Enter Product ID: ").upper()
```

```
    if pid not in inventory:
```

```
        print("Product not found.")
```

```
    return
```

```
qty = None
```

```
while qty is None:
```

```
    try:
```

```
        qty = int(input("Enter quantity to add: "))
```

```
        if qty <= 0:
```

```
            print("Quantity must be greater than 0.")
```

```
            qty = None
```

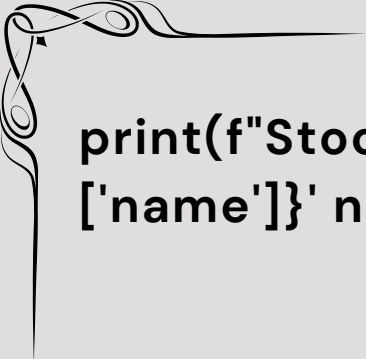
```
    except ValueError:
```

```
        print("Enter a valid number.")
```

```
inventory[pid]["quantity"] += qty
```

```
log_transaction(pid, "IN", qty)
```





```
print(f"Stocked In successful. '{inventory[pid]
['name']}' new quantity: {inventory[pid]['quantity']}")
```

```
def stock_out(): # Take out the stock
    pid = input("Enter Product ID: ").upper()
    if pid not in inventory:
        print("Product not found.")
    return
```

```
qty = None
```

```
while qty is None:
```

```
    try:
```

```
        qty = int(input("Enter quantity to remove: "))
```

```
        if qty <= 0:
```

```
            print("Quantity must be greater than 0.")
```

```
            qty = None
```

```
    except ValueError:
```

```
        print("Enter a valid number.")
```

```
if inventory[pid]["quantity"] < qty:
```

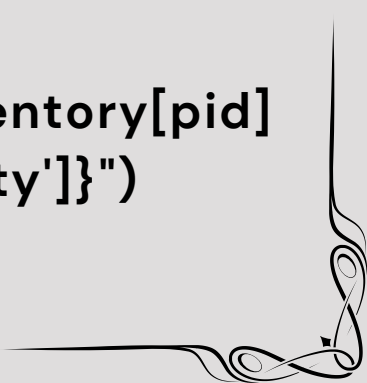
```
    print(f"Not enough stock. Available quantity:
{inventory[pid]['quantity']}")
```

```
    return
```

```
inventory[pid]["quantity"] -= qty
```

```
log_transaction(pid, "OUT", qty)
```

```
    print(f"Stock Out successful. '{inventory[pid]
['name']}' remaining: {inventory[pid]['quantity']}")
```



```

def view_inventory(): # Shows all stocks available
    if len(inventory) == 0:
        print("No products in inventory.")
        return

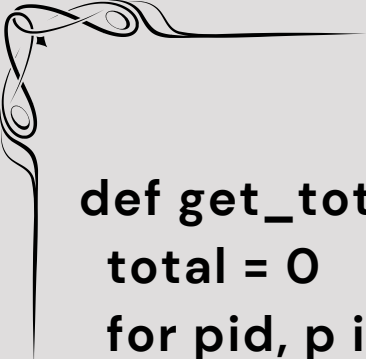
    print("\n" + "-" * 80)
        print(f"{'ID':<8}    {'Name':<20}    {'Category':<14}
{'Price':>9} {'Qty':>6} {'Reorder':>8} {'Value':>12}")
        print("-" * 80)

    for pid, p in inventory.items():
        value = p["price"] * p["quantity"]
        print(f"{'pid':<8} {'p['name']':<20} {'p['category']':<14}
{'p['price']':>9.2f}    {'p['quantity']':>6}    {'p['reorder']':>8}
{'value':>12.2f}")

    print("-" * 80)
    print(f"Total Stock Value: Rs. {get_total_value()}")

def low_stock_alert(): # Gives warning for low stocks
    print("--- Low Stock Alert ---")
    found = False
    for pid, p in inventory.items():
        if p["quantity"] <= p["reorder"]:
            found = True
            print(f" {pid} | {p['name']} | Stock: {p['quantity']} |
Reorder Level: {p['reorder']}")
    if not found:
        print("All products have sufficient stock.")

```

```
def get_total_value(): # Gives the total stock value
    total = 0
    for pid, p in inventory.items():
        total += p["price"] * p["quantity"]
    return total
```

```
def get_gst_rate(category): #Bonus task
    cat = category.strip().lower()
    if cat == "fmcg":
        return 5
    elif cat in ["apparel", "stationery"]:
        return 12
    else:
        return 18
```

```
def generate_report(): # Generates report
    if len(inventory) == 0:
        print("No products to report.")
    return
```

```
total_value = get_total_value()
categories = set()
low_stock_count = 0
top_pid = None
top_value = 0
```



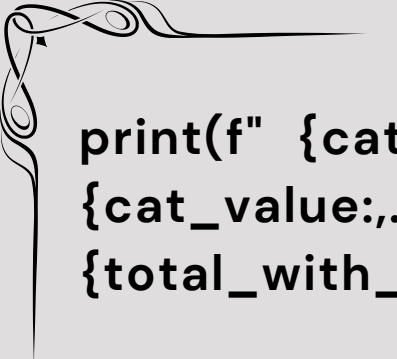
```
for pid, p in inventory.items():
    categories.add(p["category"])
    if p["quantity"] <= p["reorder"]:
        low_stock_count += 1
    val = p["price"] * p["quantity"]
    if val > top_value:
        top_value = val
        top_pid = pid
```

```
print("\n===== INVENTORY REPORT =====")
print(f"Total Products : {len(inventory)}")
print(f"Total Stock Value : Rs. {total_value:,.2f}")
print(f"Categories : {','.join(categories)}")
print(f"Low Stock Items : {low_stock_count}")
```

```
if top_pid:
    p = inventory[top_pid]
    print(f"Highest Value Item: {p['name']} (Rs.
    {p['price']} x {p['quantity']} = Rs. {top_value:,.2f})")
```

```
print("\n--- GST Calculation by Category ---") #
Bonus Task
```

```
for cat in sorted(categories):
    gst_rate = get_gst_rate(cat)
    cat_value = 0
    for pid, p in inventory.items():
        if p["category"] == cat:
            cat_value += p["price"] * p["quantity"]
    gst_amount = cat_value * gst_rate / 100
    total_with_gst = cat_value + gst_amount
```



```
print(f" {cat} | GST: {gst_rate}% | Stock Value: Rs.  
{cat_value:,.2f} | GST: Rs. {gst_amount:,.2f} | Total: Rs.  
{total_with_gst:,.2f}")
```

```
if len(transaction_log) > 0:  
    print("\n--- Transaction Log ---")  
    for entry in transaction_log:  
        print(f" {entry[3]} | Product: {entry[0]} | Type:  
{entry[1]} | Qty: {entry[2]}")  
  
    print("=====")
```

```
""Actual system below""
```

```
def show_menu():  
    print("==== INVENTORY MANAGEMENT SYSTEM  
====")  
    print(f"Products Loaded: {len(inventory)}")  
    print("1. Add Product")  
    print("2. Stock In")  
    print("3. Stock Out")  
    print("4. View Inventory")  
    print("5. Low Stock Alert")  
    print("6. Generate Report")  
    print("7. Save & Exit")
```

```
def main():  
    load_inventory()
```



```
if choice == 1:
    add_product()
elif choice == 2:
    stock_in()
elif choice == 3:
    stock_out()
elif choice == 4:
    view_inventory()
elif choice == 5:
    low_stock_alert()
elif choice == 6:
    generate_report()
elif choice == 7:
    save_inventory()
    print("Thanks for using the system")
    break
else:
    print("Invalid choice. Enter (1 to 7).")
```

```
if __name__ == "__main__":
    main()
```



Note - This code might break due to incorrect indentation while pdf making so pls use the code mentioned in .py file attached

TEST OUTPUTS -

```
Inventory loaded. Products found: 9
===== INVENTORY MANAGEMENT SYSTEM =====
Products Loaded: 9
1. Add Product
2. Stock In
3. Stock Out
4. View Inventory
5. Low Stock Alert
6. Generate Report
7. Save & Exit
```

```
Enter a choice: 1
Enter Product ID (e.g. P001): A123
Enter Product Name: Raw Pumpkin Seeds
Enter Category (Electronics/FMCG/Apparel/Stationery): fmcg
Enter Unit Price: 60
Enter Initial Quantity: 70
Enter Reorder Level: 50
Product 'Raw Pumpkin Seeds' added successfully.
```

```
Enter a choice: 2
Enter Product ID: A123
Enter quantity to add: 20
Stocked In successful. 'Raw Pumpkin Seeds' new quantity: 90
```

```
Enter a choice: 3
Enter Product ID: A123
Enter quantity to remove: 20
Stock Out successful. 'Raw Pumpkin Seeds' remaining: 70
```

Enter a choice: 4

ID	Name	Category	Price	Qty	Reorder	Value
P001	Laptop	Electronics	45000.00	5	2	225000.00
P002	USB Cable	Electronics	199.00	30	10	5970.00
P003	Headphones	Electronics	1500.00	3	5	4500.00
P004	Rice 5kg	FMCG	250.00	80	20	20000.00
P005	Cooking Oil 1L	FMCG	130.00	60	15	7800.00
P006	Soap Pack	FMCG	45.00	8	30	360.00
P007	Notebook	Stationery	60.00	150	40	9000.00
P008	Pen Set	Stationery	35.00	200	50	7000.00
P233	Patanjali Biscut	Food\\	10.00	1000	1000	10000.00
A123	Raw Pumpkin Seeds	fmcg	60.00	70	50	4200.00

Total Stock Value: Rs. 293830.0

Enter a choice: 5

--- Low Stock Alert ---

P003 | Headphones | Stock: 3 | Reorder Level: 5

P006 | Soap Pack | Stock: 8 | Reorder Level: 30

P233 | Patanjali Biscut | Stock: 1000 | Reorder Level: 1000

Enter a choice: 6

===== INVENTORY REPORT =====

Total Products : 10

Total Stock Value : Rs. 293,830.00

Categories : Stationery,Food\\,FMCG,Electronics,fmcg

Low Stock Items : 3

Highest Value Item: Laptop (Rs. 45000.0 x 5 = Rs. 225,000.00)

--- GST Calculation by Category ---

Electronics | GST: 18% | Stock Value: Rs. 235,470.00 | GST: Rs. 42,384.60 | Total: Rs. 277,854.60

FMCG | GST: 5% | Stock Value: Rs. 28,160.00 | GST: Rs. 1,408.00 | Total: Rs. 29,568.00

Food\\ | GST: 18% | Stock Value: Rs. 10,000.00 | GST: Rs. 1,800.00 | Total: Rs. 11,800.00

Stationery | GST: 12% | Stock Value: Rs. 16,000.00 | GST: Rs. 1,920.00 | Total: Rs. 17,920.00

fmcg | GST: 5% | Stock Value: Rs. 4,200.00 | GST: Rs. 210.00 | Total: Rs. 4,410.00

--- Transaction Log ---

2026-06-25 08:46:14 | Product: A123 | Type: IN | Qty: 20

2026-06-25 08:46:44 | Product: A123 | Type: OUT | Qty: 20

Enter a choice: 7

Product added & saved to inventory.txt

Thanks for using the system

TEST OUTPUTS IN BROKEN CASES -

```
Enter a choice: 2
Enter Product ID: 8
Product not found.
```

```
Enter a choice: 2
Enter Product ID: A123
Enter quantity to add: dd
Enter a valid number.
```

```
Enter a choice: 3
Enter Product ID: A123
Enter quantity to remove: Aii
Enter a valid number.
```

CODE TESTING WAS DONE SUCCESSFULLY!

More Details about the project is mentioned in the .md file attached . This pdf was made specifically to attach the code and its outputs.

PROJECT COMPLETED

